

Semana 2 — Ejemplos prácticos para clase

Guía del estudiante

Python para Ciencia de Datos | Fundamentos para Inteligencia Artificial

Todos los ejemplos usan un dataset de estudiantes, construido en la propia celda de código (no requiere subir ningún archivo), para que se puedan ejecutar de inmediato en Google Colab. Al final se incluye la versión con carga desde CSV para la parte de la sesión que sí trabaja con archivo real.

Ejemplo 0 — Dataset base de la sesión

Ejecutar esta celda primero: todos los ejemplos siguientes parten de este DataFrame.

```
import pandas as pd
import numpy as np

datos = {
    "nombre":      ["Ana", "Luis", "Marta", "Iván", "Sofía", "Pedro", "Elena",
                    "David"],
    "programa":    ["Ing. Software", "Ing. Industrial", "Ing. Software", "Ing.
                    Sistemas",
                    "Ing. Software", "Ing. Industrial", "Ing. Sistemas", "Ing.
                    Software"],
    "semestre":    [3, 5, 3, 7, 2, 5, 7, np.nan],
    "nota_final":  [4.5, 3.8, 2.9, 5.0, np.nan, 3.0, 4.1, 2.5],
    "ciudad":      ["Cúcuta", "Bogotá", "Cúcuta", np.nan, "Bogotá", "Cúcuta",
                    "Medellín", "Bogotá"]
}

df = pd.DataFrame(datos)
df
```

Qué observar: hay valores NaN puestos a propósito en `semestre`, `nota_final` y `ciudad` — así la limpieza de datos no es un ejercicio abstracto, es algo que se ve de inmediato al explorar la tabla.

Ejemplo 1 — Exploración inicial

```
df.head()          # primeras 5 filas
df.shape           # (8, 5) -> 8 filas, 5 columnas
df.info()          # tipo de dato y conteo de no-nulos por columna
df.describe()      # estadísticas de las columnas numéricas: semestre y nota_final
```

Pregunta para la clase: ¿por qué `describe()` no muestra nada sobre la columna `programa`?

Ejemplo 2 — Selección e indexación

```
# Una sola columna (devuelve una Series)
df["nota_final"]

# Varias columnas (devuelve un DataFrame)
df[["nombre", "nota_final"]]

# Por posición: primera fila, primera columna
df.iloc[0, 0]

# Por etiqueta: fila 0, columna "programa"
df.loc[0, "programa"]

# Varias filas y columnas por etiqueta
df.loc[0:2, ["nombre", "programa"]]
```

Ejemplo 3 — Filtrado booleano

```
# Estudiantes que aprobaron (nota >= 3.0)
df[df["nota_final"] >= 3.0]

# Estudiantes de Ing. Software que además aprobaron
df[(df["programa"] == "Ing. Software") & (df["nota_final"] >= 3.0)]

# Estudiantes que NO son de Bogotá
df[df["ciudad"] != "Bogotá"]
```

Ejercicio rápido: filtrar los estudiantes de semestre 5 o superior que reprobaron.

Ejemplo 4 — Conocer las categorías

```
df["programa"].unique()      # valores distintos de programa
df["programa"].value_counts() # frecuencia de cada programa
df["ciudad"].value_counts()  # frecuencia de cada ciudad (ignora los NaN por defecto)
```

Ejemplo 5 — Detectar y tratar valores nulos

```
# Detectar
df.isnull()          # tabla de True/False
```

```
df.isnull().sum()          # conteo de nulos por columna -> el primer chequeo real

# Eliminar filas con al menos un nulo
df.dropna()

# Eliminar solo si TODA la fila está vacía (no aplica aquí, pero es común verlo)
df.dropna(how="all")

# Imputar: rellenar nota_final con la media de la columna
df["nota_final"] = df["nota_final"].fillna(df["nota_final"].mean())

# Imputar: rellenar ciudad (texto) con un valor fijo, no con una media
df["ciudad"] = df["ciudad"].fillna("Sin dato")

# Imputar: rellenar semestre con la mediana (más robusta que la media ante valores atípicos)
df["semestre"] = df["semestre"].fillna(df["semestre"].median())

df
```

Punto clave: se usaron tres estrategias distintas para tres columnas distintas (media, mediana, valor fijo de texto) — esta es la idea que el entregable de la semana pide justificar por escrito.

Ejemplo 6 — Crear una columna nueva con `apply`

```
def estado_academico(nota):
    return "Aprobado" if nota >= 3.0 else "Reprobado"

df["estado"] = df["nota_final"].apply(estado_academico)
df[["nombre", "nota_final", "estado"]]
```

```
# Variante con función lambda (más compacta, útil para funciones de una sola línea)
df["nota_sobre_100"] = df["nota_final"].apply(lambda x: x * 20)
```

Ejemplo 7 — Guardar el resultado

```
df.to_csv("estudiantes_limpio.csv", index=False)
```

En Google Colab, el archivo queda en el entorno temporal de la sesión; para conservarlo hay que descargarlo ([archivos](#) > [descargar](#)) o montarlo contra Google Drive.

Ejemplo 8 — Trabajar con un archivo CSV real (para la parte final de la sesión)

Este bloque asume que subiste un archivo a Colab (panel izquierdo → *Archivos* → *Subir*).

```
df_real = pd.read_csv("nombre_del_archivo.csv")

df_real.head()
df_real.shape
df_real.info()
df_real.isnull().sum()
```

A partir de aquí, repite los ejemplos 3 a 6 sobre tu propio dataset, documentando en celdas de texto (Markdown, dentro del notebook) cada decisión de limpieza — tal como pide el entregable.

Adelanto para semanas siguientes (no resolver hoy en clase)

```
# merge – combinar dos tablas por una columna en común
programas = pd.DataFrame({
    "programa": ["Ing. Software", "Ing. Industrial", "Ing. Sistemas"],
    "facultad": ["Ingeniería", "Ingeniería", "Ingeniería"]
})
pd.merge(df, programas, on="programa", how="left")

# groupby – resumir datos por categoría
df.groupby("programa")["nota_final"].mean()
```

Ejercicios adicionales

Todos parten del mismo `df` del Ejemplo 0, salvo que se indique lo contrario. Los de NumPy son un calentamiento independiente (no dependen de `df`), como puente con la Semana 1.

A. Ejercicios para clase (guiados)

A1. Calentamiento con NumPy. Crea un array `edades = np.array([19, 22, 20, 25, 18, 23])`. Calcula, sin usar bucles `for`: a) la edad promedio, b) cuántos estudiantes son mayores de 21 (pista: `(edades > 21).sum()`), c) un nuevo array con cada edad multiplicada por 12 (edad en meses).

A2. Indexación en NumPy. Con el mismo array `edades`, obtén: el primer y el último elemento; los tres primeros; los elementos en posiciones pares (pista: `edades[::2]`); y todas las edades mayores a 20 usando una máscara booleana (`edades[edades > 20]`).

A3. Explorar un DataFrame nuevo. Construye un DataFrame con al menos 4 columnas (mínimo una numérica y una de texto) sobre un tema de tu elección (biblioteca, inventario, ventas, clima). Aplica `head()`,

`shape`, `info()` y `describe()` y explica en una frase qué te dice cada uno.

A4. Filtrado compuesto. Sobre `df`, obtén los estudiantes de semestre 5 o superior que **reprobaron** (`nota_final < 3.0`). Luego, obtén los estudiantes que **no** son de Bogotá y tienen nota mayor a 4.0.

A5. Detectar inconsistencias de captura. Crea una Series `ciudades = pd.Series(["Cúcuta", "cucuta", "CÚCUTA", "Bogotá", "bogota"])`. Usa `value_counts()` y observa cuántas categorías distintas cuenta pandas. Corrígelo con `.str.lower()` y `.str.strip()` encadenados antes de volver a contar. **Pregunta de cierre:** ¿el resultado te da exactamente las categorías reales que esperabas? Si no, ¿por qué crees que pasa?

B. Ejercicios para casa

B1. Función propia con `apply`. Sobre `df`, crea una columna `rango_edad` (o `rango_semestre`, usando la columna `semestre`) que clasifique a cada estudiante como "Inicial" (semestre 1-3), "Intermedio" (4-6) o "Avanzado" (7 o más), usando una función propia con `apply`.

B2. `groupby` básico (adelanto de la próxima semana, pero ya con las bases de hoy). Calcula el promedio de `nota_final` agrupado por `programa`. Luego calcula, además del promedio, el máximo y el mínimo por grupo (pista: `.agg(["mean", "max", "min"])`).

B3. `merge` básico (adelanto de la próxima semana). Crea una segunda tabla `ciudades_info` con columnas `ciudad` y `region` (por ejemplo: Cúcuta → Norte de Santander, Bogotá → Cundinamarca, Medellín → Antioquia). Combínala con `df` usando `pd.merge()` para agregar la columna `region` a cada estudiante. ¿Qué pasa con las filas donde `ciudad` es "Sin dato"?

B4. Limpieza guiada sobre dataset propio. Elige un dataset CSV abierto (por ejemplo, de datos.gob.es o cualquier portal de datos abiertos de tu interés) con al menos 6 columnas. Realiza el flujo completo: cargar → `head()/info()` → detectar nulos → tratarlos (justificando la estrategia por columna) → crear una columna nueva con `apply` → guardar el resultado con `to_csv()`.

B5. Ejercicio integrador (para quienes quieran un reto adicional). Sobre `df`, ordena a los estudiantes de mayor a menor `nota_final` (pista: busca en la documentación de pandas la función para ordenar un DataFrame por una columna) y muestra solo los tres mejores promedios junto con su `nombre` y `programa`.